Where compression stops paying, and why it is not where you look

Valiant Joy · The Minimalist Edge AI Challenge, Software Optimization Track Submission: `valiant_joy_int8.onnx` — the provided MobileNetV2 with 60 % of its channels removed, recovered against the provided ResNet-50 teacher, quantized to static INT8 (QDQ).

> The point at which compression stops paying is set by **how well you can recover**, not by how hard you prune — and the statistic that locates it is not the one everybody plots.

Three instruments, two of them lying

We swept one parameter, structured prune ratio, from 0 to 70 % in steps of 10, exported every point to ONNX and quantized every point. Thirteen measured variants, one table. Then we asked each of the obvious instruments where to stop, and two of them answered wrong in ways we could measure.

The mean could not see the question. Read the top of the figure as an average and it says something exciting and false: pruning 10 % of the channels *improved* accuracy, 0.9909 to 0.9944. It did not. On 5,400 validation images the 95 % interval on a difference between two variants is ±0.36 points, and everything from 0 % to 20 % sits inside it. That flat stretch is the width of our own measurement. The mean is no better at the far end: between 50 % and 60 % it loses one point, unremarkable next to the rest of the curve, while the weakest of the ten classes loses **eight and a half**. The cliff the packet asks about is invisible in the statistic most teams will plot.

Our own scorer measured the wrong thing. It ranked the variants against the rubric's weights and picked the 20 % prune. We overrode it. It normalised each criterion min–max across the sweep's own rows — and accuracy spans 2.3 points across all thirteen while file size spans more than tenfold, so min–max hands a 2.3-point spread the same dynamic range as a tenfold one. Accuracy then wins by construction rather than by the rubric, and the answer changes when you add a row to the table. A selector with that property is not measuring anything outside itself.

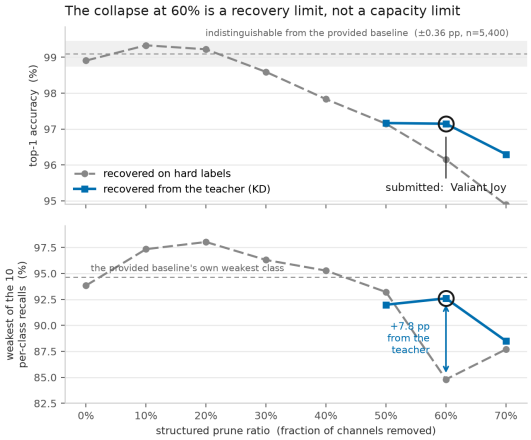
The laptop understated the prize threefold. Everything above was measured on a development machine that is not the deployment target and does not behave like one. On the CPU of a Dragonwing RB3 Gen 2 (QCS6490), the same two files give **8.94× faster inference against the laptop's 3.13×**, and 1.66× lower peak memory against 1.27×. A large out-of-order core with a deep cache absorbs the cost of a fat model and hides most of what compression buys. Choose a prune ratio by watching your own laptop and you are reading a gradient three times too shallow.

The cliff is a recovery limit

The obvious reading of that collapse at 60 % is that the architecture has run out of capacity. It has not. Everything above was recovered on hard labels. Recovered instead against the ResNet-50 the organisers provided — the soft-target route the packet suggests — the same 60 % cut comes back to 0.9715 accuracy and **0.9261 on the weakest class: seven and a half points the hard-label run threw away.** Same cut, same architecture, same quantization, four lines of configuration.

What makes that a finding rather than a tuning win is that it is **not monotonic in the damage**. Going from hard labels to teacher recovery is worth **nothing** at 50 % (0.9322 → 0.9199, slightly worse), **+7.5 points** at 60 %, and **+0.8** at 70 %. It peaks in a window: shallow enough that the network still has capacity to use a richer signal, deep enough that one correct label per image is no longer enough to rebuild from. Below that window the hard-label recovery is already at its ceiling and the teacher has nothing to add — this student was distilled from this exact teacher before we received it. Above it, there is too little left to teach.

So the deepest defensible prune is not a property of the network. It is a property of the recovery budget, and ours moved it a full step down the ladder: 2.61× parameters to 5.58×, for a change in how the fine-tuning loss is computed.



What it costs, and what we discarded

The weakest class is 0.9261, **2.05 points below the provided baseline's own 0.9466**, and two of the ten classes sit under 0.95 where the baseline has one. We are not describing that as free. The conservative alternative is real and is tagged `submission-v1`: a 40 % prune that retains 98.73 %, lifts the weakest class to 0.9528 and puts no class under 0.95 — tighter than the model we were handed. It wins the accuracy criterion and loses the other three. That trade is a judgement, so both sides ship.

Discarded, with reasons. **Unstructured pruning**: changes neither parameter count, file size nor latency — one run. **Dynamic INT8**: 12.6–16.8× *slower* than FP32 on convolutions at one thread in our own earlier measurements, and it emits operators AI Hub rejects. **Per-layer prune allocation**: measurably better than a uniform ratio at a matched size — 0.9707 against 0.9630 on the weakest class at the same parameter count — and it **cannot go deep**. Asked for 60 % of parameters it delivered 46.6 %, touching eleven of forty-nine layers: the damage budget runs out, because the parameters that are cheap to remove are a small bounded pool. Depth has to take expensive ones, which is also why the weakest class is what breaks first. **An energy measurement that disagreed with physics**: under CodeCarbon's TDP fallback the packet says energy is latency times a constant, which is testable — divide each file's Joules by its own latency and the two must agree. A three-session run gave 53.1 W and 72.3 W and was thrown out.

Every measurement

One machine throughout: 13th Gen Core i7-13620H, 16 GB, Windows 11, ONNX Runtime 1.30.0, four intra-op threads, same script and thread count for both models. Accuracy is top-1 over the 5,400 held-out images of `val_split.json` with the full confusion matrix kept, evaluated against the ONNX file. `codecarbon_mode` is `tdp_estimate` — no RAPL on this machine — so the implied-power row is the check that makes the energy *ratio* usable; the absolute Joules figure is not evidence about this model.

Measurement	Baseline	Valiant Joy	Ratio
Top-1 accuracy	99.09 %	97.15 %	**98.04 % retained**
Weakest of 10 per-class recalls	94.66 %	92.61 %	–2.05 pp
Parameters	2,236,682	400,954	**5.58×**
On-disk size, ONNX	8.51 MB	0.60 MB	**14.07×**
On-disk size, vs. the provided `.pt`	8.77 MB	0.60 MB	**14.51×**
MACs	326.2 M	79.5 M	**4.10×**
Latency, 1 thread	9.804 ms	2.679 ms	**3.66×**
Latency, 4 threads	3.501 ms	1.119 ms	**3.13×**
CPU time per inference	12.308 ms	4.646 ms	**2.65×**
Peak RAM, raw process RSS	78.85 MB	62.14 MB	**1.27×**
Energy per inference	0.2177 J	0.0709 J	**3.07×**
Implied power (energy ÷ latency)	62.2 W	63.4 W	1.02× — the consistency check
AI Hub latency, QCS6490	27.969 ms	3.127 ms	**8.94×**
AI Hub peak memory	32.33 MB	19.45 MB	**1.66×**

Peak RAM moves least and that is honest: roughly 56 MB of that process is the Python and ONNX Runtime floor, not the model. The model's own footprint drops 2.5×, but the organisers specified raw process RSS and that is what is reported.

One cost we did not pay and a judge would: recovery fine-tuning ran on a GPU, eight KD epochs at about 40 s each. The pipeline is unchanged on CPU, but that step will dominate the wall clock there, and reproducing the submitted point alone needs one prune and one recovery rather than thirteen.

```
python scripts/compress.py   --config configs/experiments/sec2026_kd.yaml \
                             --model data/sec2026/student_mobilenetv2_distilled.pt
python scripts/sec_energy.py --csv results/sec2026_kd/sweep.csv --threads 4 --seconds 30 --repeats 7
python scripts/sec_metrics.py --sweep results/sec2026_real/sweep_with_kd.csv \
                              --final kd_p60_int8 --baseline p00_fp32 --team "Valiant Joy"
```

Every figure above is a column of `tradeoff_curve.csv`, from the run those commands produce. Nothing was copied by hand.